Bài tập lập trình 1 - Phần 1: Chỉ mục không nén

Trong phần này, bạn sẽ xây dựng một hệ thống đánh chỉ mục (indexing) và truy xuất (retrieval) đơn giản cho một công cụ tìm kiếm. Bài tập này được thực hiện **cá nhân**.

Cụ thể, phần 1 bao gồm các nhiệm vụ sau:
1. **Xây dựng chỉ mục không nén (60%)** trên một tập dữ liệu các trang web từ miền stanford.edu, và hiện thực truy xuất cho các truy vấn kết hợp Boolean (Boolean conjunctive queries).
2. **Viết báo cáo (40%)** phân tích thời gian chạy, kích thước chỉ mục không nén, và cải thiện hiệu suất.

Hướng dẫn nộp bài

1\. Bài tập phải được nộp trước hạn do giảng viên quy định.

2\. Notebook sẽ tự động tạo ra các **file python** trong thư mục submission.

3\. Khi làm bài, **KHÔNG** thay đổi tên class và tên phương thức, các bài kiểm tra tự động sẽ bị lỗi nếu thay đổi.

4\. Bạn cần tải lên **phiên bản PDF** của notebook (chủ yếu dùng để chấm phần báo cáo). File PDF đặt tên theo **mã sinh viên** của bạn (ví dụ: `B22DCKH001.pdf`). Lưu ý rằng việc chuyển đổi PDF trực tiếp sẽ cắt bớt các ô code. Để có phiên bản PDF sử dụng được, trước tiên nhấp vào File > Print Preview, nó sẽ mở trong tab mới, sau đó in thành PDF bằng chức năng in của trình duyệt.

5\. Nộp file PDF báo cáo tại: [Google Drive nộp bài] (https://drive.google.com/drive/folders/1jLkXkBEpbIa53FERm5bpN8X9yK8n9bFU?usp=sharing)

6\. Bài tập được thực hiện **cá nhân**. Mọi hành vi sao chép sẽ bị xử lý theo quy định.

Thiết lập

In [1]:
```
#Tải magic tee để lưu bản sao của ô khi thực thi
%reload_ext autograding_magics
```

Thư mục `submission` sẽ chứa tất cả các file cần nộp.

In [2]:
```
import os
try:
    os.mkdir('submission')
except FileExistsError:
    pass
```

In [3]:
```
%%tee submission/imports.py

# Bạn có thể thêm các import bổ sung ở đây
import sys
import pickle as pkl
import array
import os
import timeit
import contextlib
```

```
Writing submission/imports.py
```

Tập dữ liệu (Corpus)

Tập dữ liệu mà bạn sẽ làm việc trong bài tập này chứa các trang web từ miền stanford.edu. Dữ liệu cho bài tập này có sẵn dưới dạng file .zip tại: http://web.stanford.edu/class/cs276/pa/pa1-data.zip. Đoạn code sau sẽ đặt thư mục dữ liệu vào thư mục hiện tại. Tổng kích thước tập dữ liệu khoảng 170MB.

In [4]:
```
import urllib.request
import zipfile

data_url = 'http://web.stanford.edu/class/cs276/pa/pa1-data.zip'
data_dir = 'pa1-data'

urllib.request.urlretrieve(data_url, data_dir + '.zip')

with zipfile.ZipFile(data_dir + '.zip', 'r') as zip_ref:
    zip_ref.extractall()
```

Chỉ mục sẽ được lưu trong `output_dir` và `tmp` sẽ chứa một số file tạm thời cho các chỉ mục thử nghiệm.

In [5]:
```
try:
    os.mkdir('output_dir')
except FileExistsError:
    pass
try:
    os.mkdir('tmp')
except FileExistsError:
    pass
try:
    os.mkdir('toy_output_dir')
except FileExistsError:
    pass
```

Có 10 thư mục con (đặt tên từ 0-9) trong thư mục dữ liệu.

In [6]:
```
sorted(os.listdir('pa1-data'))
```

Out[6]:
```
['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
```

Mỗi file trong thư mục con là nội dung của một trang web riêng lẻ. Bạn nên giả định rằng tên file là duy nhất trong mỗi thư mục con, nhưng không nhất thiết duy nhất giữa các thư mục con (*nghĩa là* đường dẫn đầy đủ của các file là duy nhất).

In [7]:
```
sorted(os.listdir('pa1-data/0'))[:10]
```

Out[7]:
```
['3dradiology.stanford.edu_',
 '3dradiology.stanford.edu_patient_care_Case%2520studies_AVM.html',
 '3dradiology.stanford.edu_patient_care_case_studies.html',
 '5-sure.stanford.edu_',
 '50years.stanford.edu_',
 'a3cservices.stanford.edu_awards_nominate_',
 'a3cservices.stanford.edu_facilities_',
 'a3cservices.stanford.edu_lead_',
 'aa.stanford.edu_',
 'aa.stanford.edu_about_aviation.php']
```

Tất cả thông tin HTML đã được loại bỏ khỏi các trang web, nên chúng chỉ chứa các từ phân cách bằng dấu cách. Bạn không nên tách từ (tokenize) lại. Mỗi chuỗi liên tiếp các ký tự không phải dấu cách tạo thành một token từ trong tập dữ liệu.

In [8]:
```
with open('pa1-data/0/3dradiology.stanford.edu_', 'r') as f:
    print(f.read())
```

```
3d radiology lab stanford university school of medicine stanford
school of medicine 3d and quantitative imaging in the department of
radiology search this site only stanford medical sites ways to give
find a person alumni lane library ways to give find a person about us
mission to develop and apply innovative techniques for efficient
quantitative analysis and display of medical imaging data through
interdisciplinary collaboration goals education to train physicians
and technologists locally and worldwide in the latest developments in
3d and quantitative imaging research to develop new approaches to the
exploration analysis and quantitative assesment of diagnostic images
that result in a new and or more cost effective diagnostic approaches
and b new techniques for the design and planning and monitoring of
therapy patient care to deliver valid clinically relevant
visualization and analysis of medical imaging data to the stanford
community locations richard m lucas magnetic resonance imaging center
1201 welch rd p170 stanford ca 94305 5488 650 725 8432 james h clark
center 318 campus drive s344 stanford ca 94305 5450 650 725 6862
directions to the 3qd lab local hotels you are here stanford medicine
school of medicine departments radiology 3dq laboratory navigation
for this section 3dq laboratory home education research patient care
industry about us site navigation home education overview visiting
fellowships reading list research overview research activities
research opportunities patient care overview for physicians for
patients protocol development 3dq management software case studies
testimonials industry overview infrastructure and services about us
faculty and staff history of the lab resources and equipment contact
information jobs getting to the 3dq lab stanford medicine resources
stanford medicine getting care overview find a physician find a
clinical care center stanford hospital & clinics lucile packard
children's hospital emergency research overview school of medicine
news & resources clinical trials departments institutes & centers
faculty profiles education & training overview school of medicine
programs admissions continuing medical education alumni lane medical
library community overview stanford health library community
newsletter volunteering public service & community partnerships
renewal & new building projects about us overview news careers ways
to give find a person contact us maps & directions the dean's
newsletter stanford university footer links contact us directions
members only 2009 stanford school of medicine terms of use powered by
irt
```

Bạn cũng sẽ tìm thấy một tập dữ liệu nhỏ đi kèm bài tập trong thư mục `toy-data`. Chúng ta sẽ sử dụng tập dữ liệu thử nghiệm này để kiểm tra code trước khi chạy trên tập dữ liệu đầy đủ.

In [9]:
```
toy_dir = 'toy-data'
```

Xây dựng chỉ mục không nén và truy xuất (60%)

Phần đầu tiên của bài tập này là xây dựng chỉ mục đảo (inverted index) cho tập dữ liệu, và hiện thực các truy vấn kết hợp Boolean. **Cụ thể, bạn cần hiện thực thuật toán đánh chỉ mục sắp xếp theo khối (BSBI - Blocked Sort-Based Indexing) được mô tả trong [Mục 4.2](http://nlp.stanford.edu/IR-book/pdf/04const.pdf) của sách giáo khoa.** Mặc dù chúng tôi trích dẫn một số kiến thức cơ bản từ sách giáo khoa, chúng tôi khuyến khích bạn đọc Mục 4.2 chi tiết hơn để hiểu đầy đủ thuật toán BSBI.

> Để xây dựng chỉ mục, đầu tiên chúng ta duyệt qua bộ sưu tập để thu thập tất cả các cặp term-docID. Sau đó sắp xếp các cặp với term là khóa chính và docID là khóa phụ. Cuối cùng, chúng ta tổ chức các docID cho mỗi term thành một danh sách posting và tính các thống kê như tần suất term và tần suất tài liệu. Với các bộ sưu tập nhỏ, tất cả có thể thực hiện trong bộ nhớ.

Trong bài tập này, bạn sẽ hiện thực các phương thức cho các bộ sưu tập lớn yêu cầu sử dụng bộ nhớ phụ (*tức là* đĩa).

IdMap

Trích dẫn từ Mục 4.2:

> Để việc xây dựng chỉ mục hiệu quả hơn, chúng ta biểu diễn các term dưới dạng termID (thay vì chuỗi ký tự), trong đó mỗi termID là một số thứ tự duy nhất. Chúng ta có thể xây dựng ánh xạ từ term sang termID trong quá trình xử lý bộ sưu tập. Tương tự, chúng ta cũng biểu diễn tài liệu dưới dạng docID (thay vì chuỗi ký tự).

Trước tiên, hãy xây dựng lớp trợ giúp `IdMap`, ánh xạ chuỗi sang id số (và ngược lại). Chúng ta sẽ cần điều này để ánh xạ song ánh term sang termID và doc sang docID.

Hãy điền vào các hàm `_get_str` và `_get_id` trong đoạn code sau. Giao diện duy nhất với lớp này được cung cấp bởi `__getitem__`, hàm này lấy ánh xạ chính xác tùy thuộc vào kiểu của key. (Xem [tài liệu này](https://docs.python.org/3.7/reference/datamodel.html#special-method-names) nếu bạn chưa quen với các hàm đặc biệt như `__getitem__`). Ngoài ra, để đơn giản hóa code sau này, chúng ta cũng sẽ thêm logic để thêm chuỗi nếu nó chưa tồn tại trong map.

Chúng ta sẽ sử dụng dictionary để truy cập hiệu quả từ chuỗi sang id số, và sử dụng list để truy cập (và lưu trữ) hiệu quả từ id số sang chuỗi.

Bạn cũng có thể thấy [hướng dẫn ngắn này](https://www.omkarpathak.in/2018/04/11/python-getitem-and-setitem/) hữu ích để bắt đầu với các phương thức đặc biệt (hay còn gọi là magic methods)

In [10]:
```
%%tee submission/id_map.py
class IdMap:
    """Lớp trợ giúp để lưu trữ ánh xạ từ chuỗi sang id."""
    def __init__(self):
        self.str_to_id = {}
        self.id_to_str = []

    def __len__(self):
        """Trả về số lượng term được lưu trong IdMap"""
        return len(self.id_to_str)

    def _get_str(self, i):
        """Trả về chuỗi tương ứng với id (`i`) cho trước."""
        if i < 0 or i >= len(self.id_to_str):
            raise IndexError
        return self.id_to_str[i]

    def _get_id(self, s):
        """Trả về id tương ứng với chuỗi (`s`).
        Nếu `s` chưa có trong IdMap, gán một id mới và trả về id mới đó.
        """
        if s in self.str_to_id:
            return self.str_to_id[s]
        else:
            new_id = len(self.id_to_str)
            self.str_to_id[s] = new_id
            self.id_to_str.append(s)
            return new_id

    def __getitem__(self, key):
        """Nếu `key` là số nguyên, sử dụng _get_str;
           Nếu `key` là chuỗi, sử dụng _get_id;"""
        if type(key) is int:
            return self._get_str(key)
        elif type(key) is str:
            return self._get_id(key)
        else:
            raise TypeError
```

```
Writing submission/id_map.py
```

Đảm bảo nó vượt qua các test case đơn giản sau

In [11]:
```
testIdMap = IdMap()
assert testIdMap['a'] == 0, "Không thể thêm chuỗi mới vào IdMap"
assert testIdMap['bcd'] == 1, "Không thể thêm chuỗi mới vào IdMap"
assert testIdMap['a'] == 0, "Không thể lấy id của chuỗi đã tồn tại"
assert testIdMap[1] == 'bcd', "Không thể lấy chuỗi tương ứng với id cho
trước"
try:
    testIdMap[2]
except IndexError as e:
    assert True, "Không ném IndexError cho id số ngoài phạm vi"
assert len(testIdMap) == 2
```

Từ bây giờ, chúng tôi mong đợi bạn tự viết các test case để đảm bảo các phần của chương trình hoạt động đúng như mong đợi.

Mã hóa danh sách Posting dưới dạng bytearray

Để ghi và đọc danh sách posting (docID) hiệu quả từ đĩa, chúng ta lưu chúng dưới dạng bytearray. Chúng tôi đã cung cấp lớp `UncompressedPostings` chứa các hàm encode và decode tĩnh. Trong phần tiếp theo, bạn sẽ phải hiện thực phiên bản nén với cùng giao diện.

Tham khảo:
1. https://docs.python.org/3/library/array.html
2. https://pymotw.com/3/array/#module-array

In [12]:
```
class UncompressedPostings:

    @staticmethod
    def encode(postings_list):
        """Mã hóa postings_list thành luồng byte

        Tham số
        ----------
        postings_list: List[int]
            Danh sách các docID (posting)

        Trả về
        -------
        bytes
            bytearray biểu diễn các số nguyên trong postings_list
        """
        return array.array('L', postings_list).tobytes()

    @staticmethod
    def decode(encoded_postings_list):
        """Giải mã postings_list từ luồng byte

        Tham số
        ----------
        encoded_postings_list: bytes
            bytearray biểu diễn danh sách posting đã mã hóa như đầu ra
            của hàm encode

        Trả về
        -------
        List[int]
            Danh sách docID đã giải mã từ encoded_postings_list
        """

        decoded_postings_list = array.array('L')
        decoded_postings_list.frombytes(encoded_postings_list)
        return decoded_postings_list.tolist()
```

Để minh họa cách hoạt động, hãy chạy ô sau

In [13]:
```
x = UncompressedPostings.encode([1,2,3])
print(x)
print(UncompressedPostings.decode(x))
```

```
b'\x01\x00\x00\x00\x02\x00\x00\x00\x03\x00\x00\x00'
[1, 2, 3]
```

Chỉ mục đảo trên Đĩa

> Khi bộ nhớ chính không đủ, chúng ta cần sử dụng thuật toán sắp xếp ngoại (external sorting), tức là thuật toán sử dụng đĩa. Để đạt tốc độ chấp nhận được, yêu cầu chính của thuật toán này là giảm thiểu số lần tìm kiếm ngẫu nhiên trên đĩa trong quá trình sắp xếp - đọc tuần tự trên đĩa nhanh hơn nhiều so với tìm kiếm.

Trong phần này, chúng tôi cung cấp lớp cơ sở `InvertedIndex`, lớp này sau đó sẽ được kế thừa thành `InvertedIndexWriter`, `InvertedIndexIterator` và `InvertedIndexMapper`. Mặc dù thông thường `cPickle` được sử dụng để tuần tự hóa trong Python, nó không hỗ trợ đọc và ghi một phần, đó là lý do chúng ta tự xây dựng giải pháp tùy chỉnh.

In [14]:
```
class InvertedIndex:
    """Lớp hiện thực đọc và ghi hiệu quả chỉ mục đảo lên đĩa

    Thuộc tính
    ----------
    postings_dict: Dictionary ánh xạ: termID->
(vị_trí_bắt_đầu_trong_file_index,
                                                số_posting_trong_danh_sách,

độ_dài_byte_của_danh_sách_posting)
        Đây là dictionary ánh xạ từ termID sang bộ 3 metadata
        hữu ích cho việc đọc và ghi posting trong file chỉ mục
        lên/từ đĩa. Ánh xạ này được giữ trong bộ nhớ.
        vị_trí_bắt_đầu_trong_file_index là vị trí (tính bằng byte) của danh
sách
        posting trong file chỉ mục
        số_posting_trong_danh_sách là số lượng posting (docID) trong
        danh sách posting
        độ_dài_byte_của_danh_sách_posting là độ dài mã hóa byte
        của danh sách posting

    terms: List[int]
        Danh sách các termID để ghi nhớ thứ tự mà các term và danh sách
        posting của chúng được thêm vào chỉ mục.

        Sau Python 3.7, về mặt kỹ thuật chúng ta không cần nó nữa vì dict
        trong Python là OrderedDict, nhưng vì đây là tính năng tương đối
mới,
        chúng ta vẫn duy trì tương thích ngược bằng list để theo dõi thứ tự
        chèn.
    """
    def __init__(self, index_name, postings_encoding=None, directory=''):
        """
        Tham số
        ----------
        index_name (str): Tên dùng để lưu các file liên quan đến chỉ mục
        postings_encoding: Lớp hiện thực các phương thức tĩnh để mã hóa và
            giải mã danh sách số nguyên. Mặc định là None, sẽ được thay thế
            bằng UncompressedPostings
        directory (str): Thư mục nơi các file chỉ mục sẽ được lưu
        """

        self.index_file_path = os.path.join(directory, index_name+'.index')
        self.metadata_file_path = os.path.join(directory,
index_name+'.dict')

        if postings_encoding is None:
            self.postings_encoding = UncompressedPostings
        else:
            self.postings_encoding = postings_encoding
        self.directory = directory

        self.postings_dict = {}
```

```
        self.terms = []         #Cần theo dõi thứ tự chèn các term.
                                #Không cần thiết từ Python 3.7 trở đi

    def __enter__(self):
        """Mở index_file và tải metadata khi vào context"""
        # Mở file chỉ mục
        self.index_file = open(self.index_file_path, 'rb+')

        # Tải postings dict và terms từ file metadata
        with open(self.metadata_file_path, 'rb') as f:
            self.postings_dict, self.terms = pkl.load(f)
            self.term_iter = self.terms.__iter__()

        return self

    def __exit__(self, exception_type, exception_value, traceback):
        """Đóng index_file và lưu metadata khi thoát context"""
        # Đóng file chỉ mục
        self.index_file.close()

        # Ghi postings dict và terms vào file metadata
        with open(self.metadata_file_path, 'wb') as f:
            pkl.dump([self.postings_dict, self.terms], f)
```

Vì chúng ta đang tương tác với file trên đĩa, chúng tôi cung cấp các hàm `__enter__` và `__exit__`, cho phép sử dụng câu lệnh `with` giống như khi làm việc với file IO trong Python (Tham khảo [tài liệu chính thức về context manager](https://docs.python.org/3/library/contextlib.html)).

Đây là ví dụ cách sử dụng context manager `InvertedIndexWriter`:

`python
with InvertedIndexWriter('test', directory='tmp/') as index:
Code ở đây
`

Nếu bạn chưa quen với context manager trong Python, chúng tôi khuyên bạn nên xem các hướng dẫn này ([1](https://jeffknupp.com/blog/2016/03/07/python-with-context-managers/), [2](http://arnavk.com/posts/python-context-managers/)), mặc dù không nhất thiết phải hiểu tất cả.

Đánh chỉ mục

> BSBI (i) phân chia bộ sưu tập thành các phần có kích thước bằng nhau, (ii) sắp xếp các cặp termID-docID của mỗi phần trong bộ nhớ, (iii) lưu các kết quả trung gian đã sắp xếp lên đĩa, và (iv) hợp nhất tất cả kết quả trung gian thành chỉ mục cuối cùng.

Bạn nên coi mỗi thư mục con là một khối (block), và chỉ tải một khối vào bộ nhớ tại một thời điểm khi xây dựng chỉ mục (lưu ý: bạn sẽ bị trừ điểm nếu xây dựng chỉ mục bằng cách tải các khối lớn hơn một thư mục vào bộ nhớ cùng lúc). Lưu ý rằng chúng ta đang trừu tượng hóa khái niệm block của hệ điều hành. Bạn có thể giả định rằng mỗi khối đủ nhỏ để lưu trong bộ nhớ.

Trong phần này, chúng tôi giới thiệu lớp `BSBIIndex` mà chúng ta sẽ xây dựng dần dần. Hàm `index` cung cấp khung cho BSBI và nhiệm vụ của bạn là hiện thực các hàm `parse_block`, `invert_write` và `merge` trong các phần tiếp theo.

In [15]:

```
# Không thay đổi gì ở đây, chúng sẽ bị ghi đè khi chấm điểm
class BSBIIndex:
    """
    Thuộc tính
    ----------
    term_id_map(IdMap): Để ánh xạ term sang termID
    doc_id_map(IdMap): Để ánh xạ đường dẫn tương đối của tài liệu (ví dụ
        0/3dradiology.stanford.edu_) sang docID
    data_dir(str): Đường dẫn đến dữ liệu
    output_dir(str): Đường dẫn đến thư mục chứa file chỉ mục đầu ra
    index_name(str): Tên gán cho chỉ mục
    postings_encoding: Mã hóa dùng để lưu posting.
        Mặc định (None) ngầm định sử dụng UncompressedPostings
    """
    def __init__(self, data_dir, output_dir, index_name = "BSBI",
                 postings_encoding = None):
        self.term_id_map = IdMap()
        self.doc_id_map = IdMap()
        self.data_dir = data_dir
        self.output_dir = output_dir
        self.index_name = index_name
        self.postings_encoding = postings_encoding

        # Lưu tên các chỉ mục trung gian
        self.intermediate_indices = []

    def save(self):
        """Lưu doc_id_map và term_id_map vào thư mục đầu ra"""

        with open(os.path.join(self.output_dir, 'terms.dict'), 'wb') as f:
            pkl.dump(self.term_id_map, f)
        with open(os.path.join(self.output_dir, 'docs.dict'), 'wb') as f:
            pkl.dump(self.doc_id_map, f)

    def load(self):
        """Tải doc_id_map và term_id_map từ thư mục đầu ra"""

        with open(os.path.join(self.output_dir, 'terms.dict'), 'rb') as f:
            self.term_id_map = pkl.load(f)
        with open(os.path.join(self.output_dir, 'docs.dict'), 'rb') as f:
            self.doc_id_map = pkl.load(f)

    def index(self):
        """Code đánh chỉ mục cơ sở

        Hàm này duyệt qua các thư mục dữ liệu,
        gọi parse_block để phân tích tài liệu
        gọi invert_write để đảo ngược mỗi khối và ghi vào chỉ mục mới
        sau đó lưu các id map và gọi merge trên các chỉ mục trung gian
        """
        for block_dir_relative in sorted(next(os.walk(self.data_dir))[1]):
            td_pairs = self.parse_block(block_dir_relative)
            index_id = 'index_'+block_dir_relative
```

```
            self.intermediate_indices.append(index_id)
            with InvertedIndexWriter(index_id, directory=self.output_dir,
                                     postings_encoding=
                                     self.postings_encoding) as index:
                self.invert_write(td_pairs, index)
                td_pairs = None
        self.save()
        with InvertedIndexWriter(self.index_name,
directory=self.output_dir,
                                 postings_encoding=
                                 self.postings_encoding) as merged_index:
            with contextlib.ExitStack() as stack:
                indices = [stack.enter_context(
                    InvertedIndexIterator(index_id,
                                          directory=self.output_dir,
                                          postings_encoding=
                                          self.postings_encoding))
                 for index_id in self.intermediate_indices]
                self.merge(indices, merged_index)
```

Phân tích (Parsing)

> Hàm `parse_block` phân tích tài liệu thành các cặp termID-docID và tích lũy các cặp trong bộ nhớ cho đến khi một khối có kích thước cố định đầy. Chúng ta chọn kích thước khối vừa vặn trong bộ nhớ để cho phép sắp xếp nhanh trong bộ nhớ.

Bạn nên coi mỗi thư mục con là một khối; `parse_block` nhận đường dẫn đến thư mục con (khối). Bạn nên giả định rằng tên file là duy nhất trong mỗi thư mục con, nhưng không nhất thiết duy nhất giữa các thư mục con (nghĩa là đường dẫn đầy đủ của file là duy nhất).

Lưu ý - Chúng ta đang kế thừa `BSBIIndex` từ `BSBIIndex`, đây chỉ là cách đơn giản để thêm phương thức mới vào lớp đã tồn tại. Xin đừng nhầm lẫn, nó chỉ được sử dụng như công cụ giảng dạy để chia định nghĩa lớp trong jupyter notebook và không có phép thuật nào khác ở đây.

In [16]:
```
%%tee submission/parse_block.py
class BSBIIndex(BSBIIndex):
    def parse_block(self, block_dir_relative):
        """Phân tích file văn bản đã tokenize thành các cặp termID-docID

        Tham số
        ----------
        block_dir_relative : str
            Đường dẫn tương đối đến thư mục chứa các file của khối

        Trả về
        -------
        List[Tuple[Int, Int]]
            Trả về tất cả các cặp td_pairs được trích xuất từ khối

        Nên sử dụng self.term_id_map và self.doc_id_map để lấy termID và
docID.
        Chúng được duy trì qua các lần gọi parse_block
        """
        ### Bắt đầu code của bạn
        td_pairs = []
        block_path = os.path.join(self.data_dir, block_dir_relative)
        for filename in os.listdir(block_path):
            file_path = os.path.join(block_path, filename)
            with open(file_path, 'r') as f:
                content = f.read()
            tokens = content.split()
            rel_path = os.path.join(block_dir_relative, filename)
            doc_id = self.doc_id_map[rel_path]
            for token in tokens:
                term_id = self.term_id_map[token]
                td_pairs.append((term_id, doc_id))
        return td_pairs

        ### Kết thúc code của bạn
```

```
Writing submission/parse_block.py
```

Xem hàm có hoạt động đúng trên dữ liệu thử nghiệm không.

In [17]:
```
with open('toy-data/0/fine.txt', 'r') as f:
    print(f.read())
with open('toy-data/0/hello.txt', 'r') as f:
    print(f.read())
```

```
i'm fine , thank you

hi hi
how are you ?
```

In [18]:
```
BSBI_instance = BSBIIndex(data_dir=toy_dir, output_dir = 'tmp/', index_name
= 'toy')
BSBI_instance.parse_block('0')
```

Out[18]:
```
[(0, 0),
 (1, 0),
 (2, 0),
 (3, 0),
 (4, 0),
 (5, 1),
 (5, 1),
 (6, 1),
 (7, 1),
 (4, 1),
 (8, 1)]
```

Phương thức `parse_block` có hoạt động đúng trên một khối dữ liệu thử nghiệm không? Viết một số test để đảm bảo điều đó (ví dụ, bạn nên kiểm tra rằng một từ cho trước luôn nhận được cùng một id mỗi khi nó xuất hiện).

In [ ]:
```
### Bắt đầu code của bạn

### Kết thúc code của bạn
```

Đảo ngược (Inversion)

> Khối sau đó được đảo ngược và ghi lên đĩa. Đảo ngược bao gồm hai bước. Đầu tiên, chúng ta sắp xếp các cặp termID-docID. Tiếp theo, chúng ta tập hợp tất cả các cặp termID-docID có cùng termID thành một danh sách posting, trong đó một posting đơn giản là một docID. Kết quả, một chỉ mục đảo cho khối vừa đọc, sau đó được ghi lên đĩa.

Trong phần này, chúng ta thêm hàm `invert_write` thực hiện chính xác điều này.

Tuy nhiên, trước tiên chúng ta cần hiện thực lớp `InvertedIndexWriter` . Lớp này cung cấp hàm append, giống như list, nhưng postings_list không được lưu trong bộ nhớ mà được ghi trực tiếp lên đĩa.

In [19]:
```
%%tee submission/inverted_index_writer.py
class InvertedIndexWriter(InvertedIndex):
    """ """
    def __enter__(self):
        self.index_file = open(self.index_file_path, 'wb+')
        return self

    def append(self, term, postings_list):
        """Thêm term và postings_list vào cuối file chỉ mục.

        Hàm này thực hiện ba việc:
        1. Mã hóa postings_list sử dụng self.postings_encoding
        2. Lưu metadata dưới dạng self.terms và self.postings_dict
           Lưu ý rằng self.postings_dict ánh xạ termID sang bộ 3 gồm
           (vị_trí_bắt_đầu_trong_file_index,
           số_posting_trong_danh_sách,
           độ_dài_byte_của_danh_sách_posting)
        3. Thêm luồng byte vào file chỉ mục trên đĩa

        Gợi ý: Bạn có thể thấy hữu ích khi đọc tài liệu Python I/O
        (https://docs.python.org/3/tutorial/inputoutput.html) để biết
        thông tin về cách thêm vào cuối file.

        Tham số
        ----------
        term:
            term hoặc termID là định danh duy nhất cho term
        postings_list: List[Int]
            Danh sách các docID nơi term xuất hiện
        """
        encoded = self.postings_encoding.encode(postings_list)
        start = self.index_file.tell()
        self.index_file.write(encoded)
        num_postings = len(postings_list)
        byte_len = len(encoded)
        self.postings_dict[term] = (start, num_postings, byte_len)
        self.terms.append(term)
```

```
Writing submission/inverted_index_writer.py
```

Mặc dù chưa viết các lớp đọc chỉ mục, chúng ta vẫn có thể kiểm tra hiện thực như sau. Đảm bảo bạn vượt qua các assert trước khi tiếp tục

In [20]:
```
with InvertedIndexWriter('test', directory='tmp/') as index:
    index.append(1, [2, 3, 4])
    index.append(2, [3, 4, 5])
    index.index_file.seek(0)
    assert index.terms == [1,2], "chuỗi terms không đúng"
    assert index.postings_dict == {1: (0, 3,
len(UncompressedPostings.encode([2,3,4]))),
                                   2:
(len(UncompressedPostings.encode([2,3,4])), 3,

len(UncompressedPostings.encode([3,4,5])))}, "postings_dict không đúng"
    assert UncompressedPostings.decode(index.index_file.read()) == [2, 3,
4, 3, 4, 5], "posting trên đĩa không đúng"
```

Bây giờ chúng ta hiện thực `invert_write`, nhận vào td_pairs được tạo bởi parse_block. Chúng ta sử dụng `InvertedIndexWriter` để ghi chúng lên đĩa.

In [21]:
```
%%tee submission/invert_write.py
class BSBIIndex(BSBIIndex):
    def invert_write(self, td_pairs, index):
        """Đảo ngược td_pairs thành danh sách posting và ghi chúng vào chỉ
mục cho trước

        Tham số
        ----------
        td_pairs: List[Tuple[Int, Int]]
            Danh sách các cặp termID-docID
        index: InvertedIndexWriter
            Chỉ mục đảo trên đĩa tương ứng với khối
        """
        td_pairs.sort()  # sắp xếp theo termID, rồi docID
        i = 0
        n = len(td_pairs)
        while i < n:
            term = td_pairs[i][0]
            postings = []
            while i < n and td_pairs[i][0] == term:
                postings.append(td_pairs[i][1])
                i += 1
            index.append(term, postings)
```

```
Writing submission/invert_write.py
```

Để kiểm tra trên dữ liệu thử nghiệm, chúng ta có thể đọc một khối và xem chỉ mục đảo chứa gì. Viết một số test giống như bạn đã thấy cho `InvertedIndexWriter`.

In [22]:
```
### Bắt đầu code của bạn

### Kết thúc code của bạn
```

Hợp nhất (Merging)

> Thuật toán đồng thời hợp nhất mười khối thành một chỉ mục lớn đã hợp nhất. Để thực hiện hợp nhất, chúng ta mở tất cả file khối đồng thời, và duy trì các bộ đệm đọc nhỏ cho mười khối đang đọc và một bộ đệm ghi cho chỉ mục hợp nhất cuối cùng đang ghi.

Mô hình iterator trong Python tự nhiên phù hợp với nhu cầu duy trì bộ đệm đọc nhỏ. Chúng ta có thể duyệt qua file trong khi chỉ đọc một danh sách posting tại một thời điểm từ đĩa. Chúng ta kế thừa `InvertedIndex` thành `InvertedIndexIterator` để thực hiện phần duyệt.

In [23]:
```
%%tee submission/inverted_index_iterator.py
class InvertedIndexIterator(InvertedIndex):
    """"""
    def __enter__(self):
        """Thêm initialization_hook vào hàm __enter__ của lớp cha
        """
        super().__enter__()
        self._initialization_hook()
        return self

    def _initialization_hook(self):
        """Sử dụng hàm này để khởi tạo iterator
        """
        self.current_idx = 0

    def __iter__(self):
        return self

    def __next__(self):
        """Trả về cặp (term, postings_list) tiếp theo trong chỉ mục.

        Lưu ý: Hàm này chỉ nên đọc một lượng nhỏ dữ liệu từ
        file chỉ mục. Cụ thể, bạn không nên cố gắng giữ toàn bộ
        file chỉ mục trong bộ nhớ.
        """
        if self.current_idx >= len(self.terms):
            raise StopIteration
        term = self.terms[self.current_idx]
        start, num, length = self.postings_dict[term]
        self.index_file.seek(start)
        data = self.index_file.read(length)
        postings = self.postings_encoding.decode(data)
        self.current_idx += 1
        return (term, postings)

    def delete_from_disk(self):
        """Đánh dấu chỉ mục để xóa khi thoát. Hữu ích cho chỉ mục tạm thời
        """
        self.delete_upon_exit = True

    def __exit__(self, exception_type, exception_value, traceback):
        """Xóa file chỉ mục khi thoát context cùng với
        các hàm __exit__ của lớp cha"""
        self.index_file.close()
        if hasattr(self, 'delete_upon_exit') and self.delete_upon_exit:
            os.remove(self.index_file_path)
            os.remove(self.metadata_file_path)
        else:
            with open(self.metadata_file_path, 'wb') as f:
                pkl.dump([self.postings_dict, self.terms], f)
```

```
Writing submission/inverted_index_iterator.py
```

Hãy kiểm tra bằng cách tạo chỉ mục với `InvertedIndexWriter` rồi duyệt qua nó. Viết một vài test case nhỏ để xem nó có hoạt động không. Tối thiểu, bạn nên viết một test tạo chỉ mục đảo nhỏ thủ công, ghi chúng lên đĩa bằng `InvertedIndexWriter`, sau đó dùng `InvertedIndexIterator` để duyệt qua chỉ mục đảo.

In [ ]:
```
### Bắt đầu code của bạn

### Kết thúc code của bạn
```

> Trong quá trình hợp nhất, ở mỗi lần lặp, chúng ta chọn termID thấp nhất chưa được xử lý bằng hàng đợi ưu tiên hoặc cấu trúc dữ liệu tương tự. Tất cả danh sách posting cho termID đó được đọc và hợp nhất, và danh sách đã hợp nhất được ghi lại lên đĩa. Mỗi bộ đệm đọc được nạp lại từ file khi cần thiết.

Chúng ta sẽ sử dụng `InvertedIndexIterator` để đọc và `InvertedIndexWriter` để ghi danh sách posting đã hợp nhất.

Lưu ý rằng chúng ta đã mở mỗi chỉ mục đảo bằng câu lệnh `with`, nhưng bây giờ cần làm điều đó theo chương trình. Bạn có thể thấy chúng tôi đã sử dụng `contextlib` ([tài liệu] (https://docs.python.org/3/library/contextlib.html#contextlib.ExitStack)) trong hàm `index` đã cung cấp của lớp `BSBIIndex` cơ sở. Nhiệm vụ của bạn là **viết logic** để hợp nhất các đối tượng `InvertedIndexIterator` đã mở và ghi từng danh sách posting vào một đối tượng `InvertedIndexWriter` duy nhất.

Vì chúng ta biết các posting đã được sắp xếp, chúng ta có thể hợp nhất chúng theo thứ tự trong thời gian tuyến tính. Thực tế `heapq` ([tài liệu] (https://docs.python.org/3.0/library/heapq.html)) là module chuẩn Python cung cấp hiện thực thuật toán hàng đợi heap. Cụ thể, nó chứa hàm tiện ích `heapq.merge` hợp nhất nhiều đầu vào đã sắp xếp thành một đầu ra đã sắp xếp và trả về iterator trên các giá trị đã sắp xếp. Không chỉ hữu ích cho hợp nhất danh sách posting, mà còn cho hợp nhất các chỉ mục đảo.

Để giúp bạn bắt đầu với hàm `heapq.merge`, chúng tôi cung cấp ví dụ sử dụng. Hai danh sách chứa động vật/chim được sắp xếp theo tuổi thọ trung bình. Chúng ta muốn hợp nhất hai danh sách.

In [24]:
```
import heapq
animal_lifespans = [('Giraffe', 28),
                    ('Rhinoceros', 40),
                    ('Indian Elephant', 70),
                    ('Golden Eagle', 80),
                    ('Box turtle', 123)]

tree_lifespans = [('Gray Birch', 50),
                  ('Black Willow', 70),
                  ('Basswood', 100),
                  ('Bald Cypress', 600)]

lifespan_lists = [animal_lifespans, tree_lifespans]

for merged_item in heapq.merge(*lifespan_lists, key=lambda x: x[1]):
    print(merged_item)
```

```
('Giraffe', 28)
('Rhinoceros', 40)
('Gray Birch', 50)
('Indian Elephant', 70)
('Black Willow', 70)
('Golden Eagle', 80)
('Basswood', 100)
('Box turtle', 123)
('Bald Cypress', 600)
```

Lưu ý cách sử dụng `*` để giải nén `lifespan_lists` thành đối số và hàm `lambda` để biểu diễn khóa sắp xếp. Nếu chưa quen, bạn có thể xem tài liệu ([giải nén danh sách](https://docs.python.org/3/tutorial/controlflow.html#unpacking-argument-lists), [biểu thức lambda](https://docs.python.org/3/tutorial/controlflow.html#lambda-expressions)) hoặc hướng dẫn ([giải nén danh sách](https://www.geeksforgeeks.org/packing-and-unpacking-arguments-in-python/), [biểu thức lambda](https://www.afternerd.com/blog/python-lambdas/))

In [25]:
```
%%tee submission/merge.py

import heapq
class BSBIIndex(BSBIIndex):
    def merge(self, indices, merged_index):
        """Hợp nhất nhiều chỉ mục đảo thành một chỉ mục duy nhất

        Tham số
        ----------
        indices: List[InvertedIndexIterator]
            Danh sách các đối tượng InvertedIndexIterator, mỗi đối tượng
            đại diện cho một chỉ mục đảo có thể duyệt cho một khối
        merged_index: InvertedIndexWriter
            Một đối tượng InvertedIndexWriter để ghi từng danh sách
            posting đã hợp nhất
        """
        merged = heapq.merge(*indices, key=lambda x: x[0])  # x[0] là term
        current_term = None
        current_postings = []
        for term, postings in merged:
            if term != current_term:
                if current_term is not None:
                    merged_index.append(current_term, current_postings)
                current_term = term
                current_postings = postings
            else:
                current_postings.extend(postings)
        if current_term is not None:
            merged_index.append(current_term, current_postings)
```

```
Writing submission/merge.py
```

Trước tiên đảm bảo nó chạy không lỗi trên dữ liệu thử nghiệm

In [26]:
```
BSBI_instance = BSBIIndex(data_dir=toy_dir, output_dir = 'toy_output_dir',
)
BSBI_instance.index()
```

Bây giờ hãy đánh chỉ mục cho toàn bộ tập dữ liệu

In [ ]:
```
BSBI_instance = BSBIIndex(data_dir='pa1-data', output_dir = 'output_dir', )
BSBI_instance.index()
```

Nếu bạn gặp lỗi chỉ ở phần hợp nhất, bạn có thể sử dụng đoạn code sau để gỡ lỗi.

In [ ]:
```
BSBI_instance = BSBIIndex(data_dir='pa1-data', output_dir = 'output_dir', )
BSBI_instance.intermediate_indices = ['index_'+str(i) for i in range(10)]
with InvertedIndexWriter(BSBI_instance.index_name,
directory=BSBI_instance.output_dir,
postings_encoding=BSBI_instance.postings_encoding) as merged_index:
    with contextlib.ExitStack() as stack:
        indices = [stack.enter_context(InvertedIndexIterator(index_id,
directory=BSBI_instance.output_dir,
postings_encoding=BSBI_instance.postings_encoding)) for index_id in
BSBI_instance.intermediate_indices]
        BSBI_instance.merge(indices, merged_index)
```

Truy xuất kết hợp Boolean (Boolean conjunctive retrieval)

Chúng ta sẽ hiện thực hàm `retrieve` cho BSBIIndex, hàm này nhận một chuỗi truy vấn gồm các token phân cách bằng dấu cách, trả về danh sách tài liệu chứa mỗi token trong truy vấn. Tuy nhiên, chúng ta không muốn duyệt qua chỉ mục hay tải toàn bộ chỉ mục để tìm các term liên quan.

Trước tiên, chúng ta sẽ hiện thực `InvertedIndexMapper` kế thừa `InvertedIndex` để thêm chức năng truy xuất posting tương ứng với một term cụ thể bằng cách seek đến vị trí đó trong file.

In [ ]:
```
%%tee submission/inverted_index_mapper.py
class InvertedIndexMapper(InvertedIndex):
    def __getitem__(self, key):
        return self._get_postings_list(key)

    def _get_postings_list(self, term):
        """Lấy danh sách posting (các docId) cho `term`.

        Hàm này không nên duyệt qua file chỉ mục.
        Tức là, nó chỉ cần đọc các byte từ file chỉ mục
        tương ứng với danh sách posting cho term được yêu cầu.
        """
        if term not in self.postings_dict:
            raise KeyError
        start, num, length = self.postings_dict[term]
        self.index_file.seek(start)
        data = self.index_file.read(length)
        return self.postings_encoding.decode(data)
```

Viết một vài test để kiểm tra hiện thực `_get_postings_list` của bạn.

In [ ]:
```
### Bắt đầu code của bạn

### Kết thúc code của bạn
```

Bây giờ chúng ta có thể lấy danh sách posting tương ứng với các term truy vấn, chúng ta muốn giao chúng. Tuy nhiên, tương tự như hợp nhất, chúng ta có thể sử dụng tính chất đã sắp xếp của các danh sách này để giao chúng rất hiệu quả. Ở đây chúng ta sẽ hiện thực `sorted_intersect` nhận hai danh sách đã sắp xếp và trả về giao đã sắp xếp của các phần tử.

In [ ]:
```
%%tee submission/sorted_intersect.py
def sorted_intersect(list1, list2):
    """Giao hai danh sách đã sắp xếp (tăng dần) và trả về kết quả đã sắp
xếp

    Tham số
    ----------
    list1: List[Comparable]
    list2: List[Comparable]
        Các danh sách đã sắp xếp cần giao

    Trả về
    -------
    List[Comparable]
        Giao đã sắp xếp
    """
    i, j = 0, 0
    result = []
    while i < len(list1) and j < len(list2):
        if list1[i] == list2[j]:
            result.append(list1[i])
            i += 1
            j += 1
        elif list1[i] < list2[j]:
            i += 1
        else:
            j += 1
    return result
```

Viết một vài test case đơn giản để đảm bảo nó hoạt động chính xác

In [ ]:
```
### Bắt đầu code của bạn

### Kết thúc code của bạn
```

Bây giờ chúng ta sẵn sàng viết hàm `retrieve` sử dụng `InvertedIndexMapper` và `sorted_intersect`

In [ ]:
```
%%tee submission/retrieve.py
class BSBIIndex(BSBIIndex):
    def retrieve(self, query):
        """Truy xuất các tài liệu tương ứng với truy vấn kết hợp

        Tham số
        ----------
        query: str
            Danh sách các token truy vấn phân cách bằng dấu cách

        Kết quả
        ------
        List[str]
            Danh sách tài liệu đã sắp xếp chứa mỗi token truy vấn.
            Trả về rỗng nếu không tìm thấy tài liệu.

        KHÔNG nên ném lỗi cho các term không có trong tập dữ liệu
        """
        if len(self.term_id_map) == 0 or len(self.doc_id_map) == 0:
            self.load()

        tokens = query.split()
        term_ids = []
        for token in tokens:
            if token in self.term_id_map.str_to_id:
                term_ids.append(self.term_id_map[token])
            else:
                return []  # term không tồn tại

        with InvertedIndexMapper(self.index_name,
directory=self.output_dir,
                                 postings_encoding=self.postings_encoding)
as index:
            postings_lists = []
            for tid in term_ids:
                try:
                    pl = index[tid]
                    postings_lists.append((tid, pl))
                except KeyError:
                    return []
            # sắp xếp theo độ dài danh sách tăng dần để tối ưu giao
            postings_lists.sort(key=lambda x: len(x[1]))
            result = postings_lists[0][1]
            for _, pl in postings_lists[1:]:
                result = sorted_intersect(result, pl)
                if not result:
                    break
            # chuyển docID thành đường dẫn
            doc_paths = [self.doc_id_map[doc_id] for doc_id in result]
            return doc_paths
```

Hãy kiểm tra xem chỉ mục của chúng ta có hoạt động trên tập dữ liệu thực với một truy vấn đơn giản không

In [ ]:
```
BSBI_instance = BSBIIndex(data_dir='pa1-data', output_dir = 'output_dir', )
BSBI_instance.retrieve('boolean retrieval')
```

Bạn cũng có thể kiểm tra xem một trong các trang đó có thực sự chứa các term truy vấn không bằng cách đọc file như sau

In [ ]:
```
with open("pa1-data/1/cs276.stanford.edu_", 'r') as f:
    print(f.read())
```

Hãy kiểm tra trên các truy vấn dev để xem nó có hoạt động đúng không.

In [ ]:
```
for i in range(1, 9):
    with open('dev_queries/query.' + str(i)) as q:
        query = q.read()
        my_results = [os.path.normpath(path) for path in
BSBI_instance.retrieve(query)]
        with open('dev_output/' + str(i) + '.out') as o:
            reference_results = [os.path.normpath(x.strip()) for x in
o.readlines()]
            assert my_results == reference_results, "Kết quả KHÔNG khớp cho
truy vấn: "+query.strip()
        print("Kết quả khớp cho truy vấn:", query.strip())
```

Nếu một assert bị lỗi, bạn có thể so sánh sự khác biệt trong đầu ra như sau

In [ ]:
```
set(my_results) - set(reference_results)
```

In [ ]:
```
set(reference_results) - set(my_results)
```

Đảm bảo bạn viết các truy vấn riêng để kiểm tra tất cả các trường hợp đặc biệt như term chưa từng xuất hiện trong tập dữ liệu, hoặc term rất phổ biến có thể làm chậm quá trình hợp nhất.

Phân bổ điểm

Để đảm bảo chỉ mục được xây dựng đúng, bạn sẽ được kiểm tra trên 20 truy vấn kết hợp Boolean gồm một hoặc nhiều term. Cho mỗi truy vấn, bạn sẽ nhận 1.5% điểm cuối cùng cho mỗi truy vấn trả lời đúng, tổng cộng 30% điểm. 8 trong số đó sẽ giống các truy vấn dev đã cung cấp.

Lưu ý rằng chúng tôi sẽ chạy bài nộp của bạn trên VM yếu (để mô phỏng điều kiện thực tế với kích thước khối lớn hơn và nhiều bộ nhớ hơn) với khoảng 750 MB bộ nhớ khả dụng. Nếu chương trình sử dụng nhiều hơn, nó sẽ bị kill bởi VM. Bạn nên đảm bảo trước thời hạn rằng bài nộp hoạt động và vượt qua các bài kiểm tra và truy vấn hiển thị khi nộp lên autograder gradescope.

Ngoài ra, chúng tôi cũng sẽ chạy một số bài kiểm tra để xem các file chỉ mục trung gian và cuối cùng có khớp với hiện thực tham chiếu không, cũng như một số unit test để kiểm tra tính đúng đắn của từng hàm yêu cầu hiện thực. Lưu ý do ràng buộc thứ tự, chỉ có đúng một đầu ra chính xác. Bạn sẽ nhận 15% điểm còn lại dựa trên đó. Phân bổ như sau:

1. `IdMap` - 2%
2. `parse_block` - 2%
3. `InvertedIndexWriter` - 1%
4. `invert_write` - 2%
5. `InvertedIndexIterator` - 1%
6. `merge` - 2%
7. `InvertedIndexMapper` - 1%
8. `sorted_intersect` - 2%
9. `retrieve` - 2%

Chúng tôi cũng sẽ thu thập thống kê thời gian về thời gian đánh chỉ mục, kích thước chỉ mục và lượng bộ nhớ sử dụng khi xây dựng chỉ mục.
Cuối cùng, chúng tôi sẽ kiểm tra code, các chỉ mục được tạo và trừ điểm theo:
1. 5% nếu hàm `retrieve` tải toàn bộ chỉ mục vào bộ nhớ thay vì chỉ tải danh sách posting của các term truy vấn.
2. 5% nếu hàm `merge` tải toàn bộ chỉ mục trung gian hoặc giữ toàn bộ chỉ mục đã hợp nhất trong bộ nhớ.
3. 4% nếu bạn không sắp xếp các term truy vấn theo độ dài danh sách posting trong hàm `retrieve`.
4. 4% nếu bạn sử dụng phép toán tập hợp built-in (không tận dụng tính chất đã sắp xếp của danh sách posting) khi hợp nhất danh sách posting trong hàm `merge` hoặc giao danh sách đã sắp xếp trong hàm `sorted_intersect`.
5. 5% nếu thống kê thời gian của thuật toán truy xuất (hàm `retrieve`) nằm ngoài chuẩn (có thể xảy ra nếu bạn duyệt qua chỉ mục thay vì seek đến các term truy vấn).
6. 5% nếu kích thước bộ nhớ của chỉ mục được tạo nằm ngoài chuẩn

Tổng điểm tối đa cho phần này là 45%.

Báo cáo (40%)

In [ ]:
```
%timeit BSBI_instance.retrieve('boolean retrieval')
```

`q1` - Truy vấn chậm hơn (2%)

Làm thế nào bạn có thể thay thế một term trong truy vấn để nó chạy chậm hơn? Đo thời gian truy vấn và xem nó có đúng như giả thuyết của bạn không. Chúng ta sẽ gọi truy vấn này là `q1`

In [ ]:
```
### Bắt đầu code của bạn
ư
q1 = 'the'
%timeit BSBI_instance.retrieve(q1)
### Kết thúc code của bạn
```

Lý do đằng sau sự thay đổi là gì? Tại sao hàm retrieve chạy chậm hơn?
> Term "the" là một từ rất phổ biến, xuất hiện trong hầu hết các tài liệu. Do đó, danh sách posting của nó rất dài (có thể lên tới hàng chục nghìn docID). Việc đọc danh sách dài từ đĩa và giải mã tốn nhiều thời gian hơn so với một term hiếm như "boolean" hoặc "retrieval". Ngoài ra, nếu truy vấn chỉ có một term, không có phép giao, thời gian truy xuất tỉ lệ thuận với độ dài danh sách posting.

`q2` - Truy vấn chậm gấp đôi (2%)

Bạn có thể làm cho truy xuất chậm gấp đôi không? Xây dựng một truy vấn chạy chậm gấp đôi so với truy vấn trước (`q1`). Chúng ta sẽ gọi truy vấn này là `q2`

In [ ]:
```
### Bắt đầu code của bạn
q2 = 'and'
%timeit BSBI_instance.retrieve(q2)
### Kết thúc code của bạn
```

Nguyên tắc chung đằng sau việc xây dựng truy vấn chạy chậm gấp đôi là gì?

> Chọn term có tần suất tài liệu (document frequency) cao hơn, tức là danh sách posting dài hơn. Thời gian đọc và giải mã tỉ lệ thuận với độ dài danh sách. Nếu danh sách của "and" dài gấp đôi "the", thời gian sẽ gần gấp đôi. Tuy nhiên, cần lưu ý các yếu tố khác như tốc độ đọc đĩa, bộ đệm, v.v.

`q3` - Yếu tố nhiễu (2%)

Truy vấn chạy chậm gấp đôi (`q2`) có thể khác truy vấn trước (`q1`) ở nhiều điểm. Do đó, sự chậm lại có thể do bất kỳ sự khác biệt nào gây ra.

Viết một hoặc nhiều truy vấn để chứng minh rằng nguyên tắc chung đã nêu là nguyên nhân duy nhất gây chậm và không có sự khác biệt nào khác.

In [ ]:
```
### Bắt đầu code của bạn
q3a = 'the'   # phổ biến
q3b = 'and'   # phổ biến hơn
q3c = 'of'    # cũng phổ biến
# 2. So sánh với term hiếm
q3d = 'boolean
### Kết thúc code của bạn
```

Bạn đã loại bỏ được những yếu tố nào là nguyên nhân gây chậm? Mô tả cách các truy vấn loại bỏ các yếu tố đó.

> Yếu tố độ dài term: Các term như "the" và "and" đều ngắn, không ảnh hưởng đến thời gian đọc. Yếu tố loại term: Đều là stopword, không có khác biệt về mặt ngữ nghĩa. Yếu tố vị trí trên đĩa: Chọn các term có posting list nằm ở các vùng khác nhau của file index, nhưng thời gian đọc vẫn tỉ lệ với độ dài, cho thấy seek time không đáng kể so với read time. Yếu tố bộ đệm: Chạy nhiều lần để ổn định, kết quả lặp lại.

Có yếu tố nào bạn không thể loại bỏ không? Bạn có thể thay đổi `q2` để nó vẫn chậm gấp đôi `q1` nhưng không còn yếu tố nhiễu không? Nếu không, bạn có cần xem xét lại nguyên tắc chung cho sự chậm lại trong `q2` không?

`q4` - Thêm term mới (2%)

Xây dựng truy vấn mới bằng cách thêm một term vào `q2` để nó nhanh hơn `q2` (mặc dù có thể chậm hơn `q1` nhiều). Chúng ta gọi nó là `q4`

In [ ]:
```
### Bắt đầu code của bạn
q4 = 'and boolean'
%timeit BSBI_instance.retrieve(q4)
### Kết thúc code của bạn
```

Bạn đã làm thế nào? Nguyên tắc đằng sau việc tăng tốc là gì?

> Thêm một term hiếm ("boolean") có posting list rất ngắn. Khi thực hiện phép giao, thuật toán sorted_intersect sẽ duyệt qua danh sách ngắn trước (nếu được sắp xếp) hoặc theo thứ tự tối ưu, kết quả là chỉ cần đọc danh sách ngắn và kiểm tra trên danh sách dài, giảm thời gian đáng kể so với đọc toàn bộ danh sách dài. Kết quả trả về rất ít tài liệu, thậm chí có thể rỗng.

`q5` - Ảnh hưởng của việc xáo trộn (2%)

Xáo trộn các term trong `q4` và đo thời gian

In [ ]:
```
### Bắt đầu code của bạn
q4 = 'and boolean'
q5 = 'boolean and'  # đảo thứ tự
%timeit BSBI_instance.retrieve(q4)
%timeit BSBI_instance.retrieve(q5)
### Kết thúc code của bạn
```

Bạn có nhận thấy sự khác biệt *đáng kể* giữa thời gian của `q4` và `q5` không? Tại sao bạn không mong đợi thời gian thay đổi quá nhiều?

> Trong code retrieve, chúng ta đã sắp xếp các term theo độ dài posting list tăng dần trước khi giao. Do đó, dù thứ tự truy vấn thế nào, thuật toán vẫn xử lý term ngắn trước. Vì vậy thời gian không thay đổi. Nếu không có bước sắp xếp đó, thứ tự có thể ảnh hưởng (giao list dài trước sẽ chậm hơn). Nhưng ở đây chúng ta đã tối ưu

Lưu ý rằng các phép đo thời gian này không hoàn hảo nhưng bạn sẽ có thể thấy sự khác biệt đáng kể ở những nơi mong đợi mà không cần quá nhiều nỗ lực

Kích thước chỉ mục (6%)

Trong phần này, chúng ta sẽ xem xét kích thước chỉ mục trên đĩa. Mỗi phần con chiếm 2% bài tập. Chúng tôi sẽ chấm cả code phân tích và câu trả lời viết.

Chỉ mục không nén (2%)

Hãy xem kích thước file chỉ mục đã hợp nhất (`.index`)

In [ ]:
```
print("Kích thước chỉ mục", os.path.getsize("output_dir/BSBI.index"),
'bytes')
```

Đoạn code trên truy vấn hệ thống file để tính kích thước chỉ mục. Hãy chứng minh rằng bạn có thể tính cùng kết quả bằng cách nhìn trực tiếp vào đối tượng `postings_dict` . Cụ thể, viết một dòng code Python để tính kích thước file chỉ mục (tính bằng byte) sử dụng postings dict.

Gợi ý: Bạn nên sử dụng kích thước từ (word size) (tính bằng byte) và thông tin kích thước lưu trong `postings_dict` .

In [ ]:
```
with open('output_dir/BSBI.dict', 'rb') as f:
    postings_dict, terms = pkl.load(f)

total_bytes = 0
for term, (start, num, length) in postings_dict.items():
    total_bytes += length
print("Kích thước tính từ postings_dict:", total_bytes, "bytes")
```

Bạn đã tính kích thước dự kiến dựa trên thông tin lưu trong postings dictionary như thế nào? Kích thước mỗi số nguyên biểu diễn posting là bao nhiêu?

> Mỗi posting (docID) được lưu dưới dạng số nguyên không dấu 8 byte (do dùng array('L') trong UncompressedPostings). Tổng số posting là tổng num trong postings_dict. Tuy nhiên, ở đây ta cộng trực tiếp length (độ dài byte) của mỗi danh sách. length chính là số posting * 8. Do đó tổng byte là tổng các length. Kết quả này khớp với os.path.getsize của file index.

Giảm kích thước chỉ mục không nén (2%)

Một cách giảm kích thước chỉ mục không nén là đơn giản sử dụng ít bit hơn cho mỗi posting. Số bit tối thiểu cần thiết để biểu diễn posting (tức docId) cho bộ sưu tập tài liệu này là bao nhiêu? Trong ô sau, thu thập các thống kê giúp bạn xác định câu trả lời

In [ ]:
```
# Đếm số tài liệu
num_docs = len(BSBI_instance.doc_id_map)
min_bits = (num_docs - 1).bit_length()
print("Số tài liệu:", num_docs)
print("Số bit tối thiểu:", min_bits)
```

Nếu mỗi posting được biểu diễn bằng số bit bạn tính được ở trên, kích thước dự kiến của chỉ mục không nén là bao nhiêu? Bạn đã tính như thế nào?
Bạn có thể dùng ô trước để tính toán thêm

> Giả sử số tài liệu là 50,000, thì cần 16 bit (vì 2^16 = 65536 > 50000). Kích thước chỉ mục dự kiến nếu dùng 16 bit cho mỗi posting: Tổng số posting trong toàn bộ chỉ mục (giả sử là N). N có thể tính bằng tổng num trong postings_dict. Nếu mỗi posting dùng 2 byte (16 bit) thay vì 8 byte, kích thước giảm 4 lần. Kích thước hiện tại là total_bytes, kích thước mới ≈ total_bytes / 4. Ví dụ: nếu total_bytes = 100 MB, thì kích thước mới ≈ 25 MB.

Cải thiện hiệu suất (9%)

Trong phần này, bạn sẽ viết một vài câu (khoảng 5 câu) để trả lời mỗi câu hỏi sau. Mỗi câu hỏi chiếm 3% bài tập.

Câu 1

Trong bài tập này, chúng tôi yêu cầu bạn sử dụng mỗi thư mục con như một khối và xây dựng chỉ mục cho từng khối một. Bạn có thể thảo luận về sự đánh đổi giữa các kích thước khối khác nhau không? Có chiến lược chung nào khi làm việc với bộ nhớ giới hạn nhưng muốn giảm thiểu thời gian đánh chỉ mục không?

> Kích thước khối ảnh hưởng đến cả bộ nhớ và thời gian. Khối nhỏ hơn (ít tài liệu hơn) giảm bộ nhớ sử dụng trong quá trình xử lý từng khối, nhưng tăng số lượng khối, dẫn đến nhiều lần đọc/ghi trung gian và công đoạn hợp nhất phức tạp hơn. Ngược lại, khối lớn hơn tận dụng được bộ nhớ tốt hơn, giảm số lần đọc/ghi và thời gian hợp nhất, nhưng có thể vượt quá bộ nhớ khả dụng. Chiến lược chung: Chọn kích thước khối sao cho vừa với bộ nhớ (ví dụ, 80% bộ nhớ) để tối đa hóa hiệu quả. Nếu bộ nhớ rất hạn chế, cần chia nhỏ khối, nhưng có thể áp dụng kỹ thuật nén hoặc sử dụng bộ nhớ ngoài hiệu quả hơn. Ngoài ra, có thể tối ưu bằng cách xử lý song song các khối nếu có nhiều CPU.

Câu 2

Có phần nào trong chương trình đánh chỉ mục của bạn hạn chế khả năng mở rộng với tập dữ liệu lớn hơn không?

> Một số hạn chế: IdMap lưu toàn bộ ánh xạ term-docID trong bộ nhớ. Với tập dữ liệu rất lớn (hàng triệu term), dictionary và list có thể chiếm nhiều bộ nhớ và gây chậm. postings_dict cũng lưu metadata của mọi term trong bộ nhớ, tương tự. Quá trình hợp nhất dùng heapq.merge cần mở nhiều file cùng lúc, giới hạn bởi số file descriptor của hệ thống. Nếu số khối quá lớn, việc hợp nhất nhiều luồng có thể chậm do I/O và sắp xếp. Thuật toán giao hai list trong retrieve là O(n+m) nhưng nếu list rất dài, vẫn có thể chậm; tuy nhiên, đó là tối ưu cho tập đã sắp xếp.

Câu 3

Mô tả các phần khác của quá trình đánh chỉ mục mà bạn có thể tối ưu hóa về thời gian đánh chỉ mục/khả năng mở rộng và thời gian truy xuất.

> Nén danh sách posting: Dùng mã hóa delta (gap) và Variable Byte Encoding hay Golomb để giảm kích thước lưu trữ và tăng tốc đọc (ít byte hơn). Điều này giảm I/O và cải thiện thời gian truy xuất. Bộ đệm (caching): Cache các danh sách posting thường dùng trong bộ nhớ để truy xuất nhanh. Song song hóa: Xử lý nhiều khối cùng lúc nếu có nhiều CPU, và hợp nhất song song. Tối ưu hóa sắp xếp: Dùng thuật toán sắp xếp ngoại hiệu quả hơn nếu bộ nhớ hạn chế. Chọn thứ tự giao thông minh: Trong retrieve, sắp xếp term theo độ dài posting list tăng dần để giảm thời gian giao. Lưu metadata riêng: Tách riêng term dictionary khỏi posting để có thể tìm nhanh term mà không cần đọc toàn bộ. Đánh chỉ mục theo từng phần (incremental indexing): Cho phép cập nhật chỉ mục mà không cần xây dựng lại toàn bộ.

Nộp báo cáo

Bây giờ bạn sẵn sàng nộp báo cáo. Xuất notebook thành file PDF đặt tên theo **mã sinh viên** (ví dụ: `B21DCCN001.pdf`) và tải lên [Google Drive nộp bài](https://drive.google.com/drive/folders/1jLkXkBEpbIa53FERm5bpN8X9yK8n9bFU?usp=sharing).

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.